1) ~~Pgm~~ univariant, Bivariant and multivariant Regression

Aim:-

To implement Python Program using univariate, bivariant and multivariate Regression for the given iris dataset

Algorithm:-

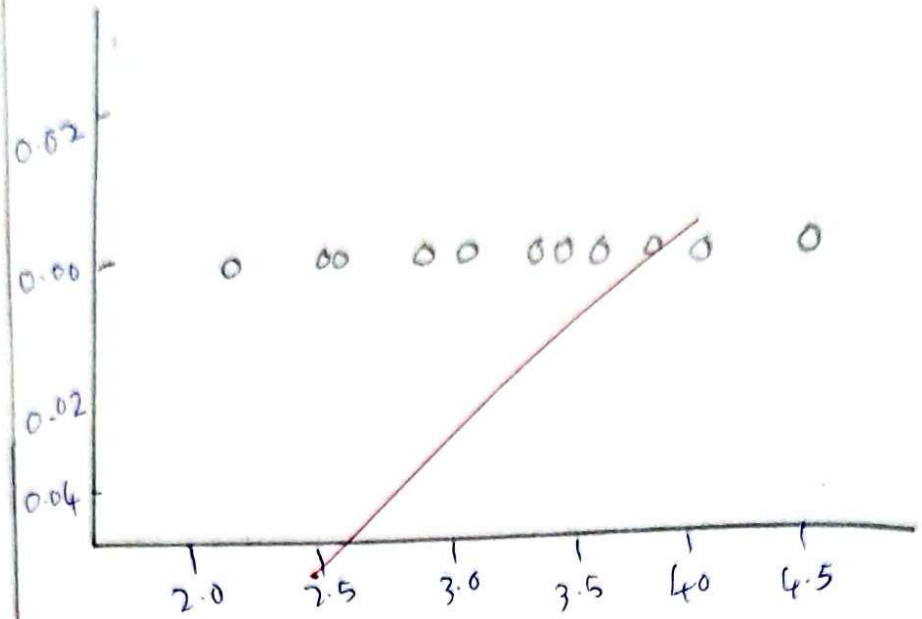
* Import Pandas, numpy, matplotlib.pyplot libraries
* Read the data set using Pandas 'read_CSV'
* Extract the Independent variable X and Dependent variable Y from the dataset
* Reshape X and Y to be 2D array if needed

Univariate Regression:-

* here we only Independent variable
* Fit a Linear Regression model to the data using numpy's Polyfit function.
* make Prediction using the model
* calculate the R-Squared value to evaluate model's performance

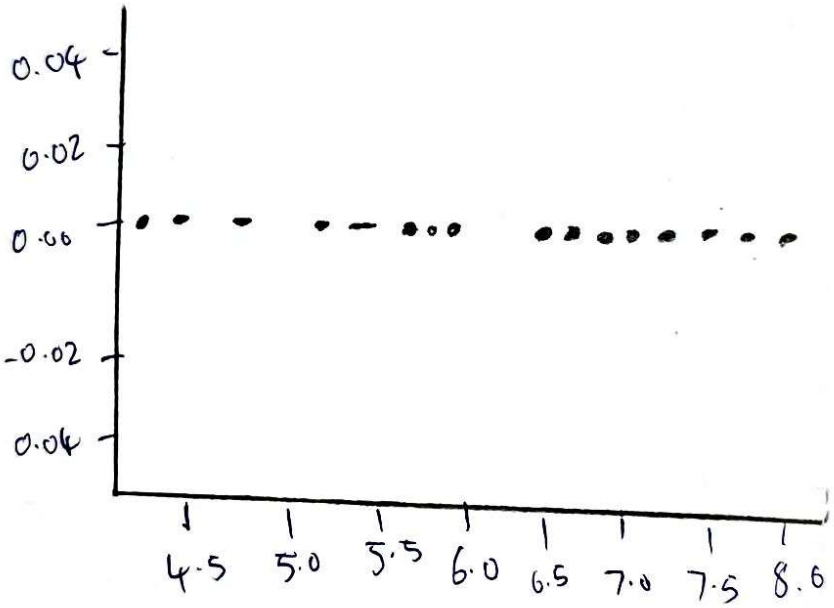
Bivariat Regression:-

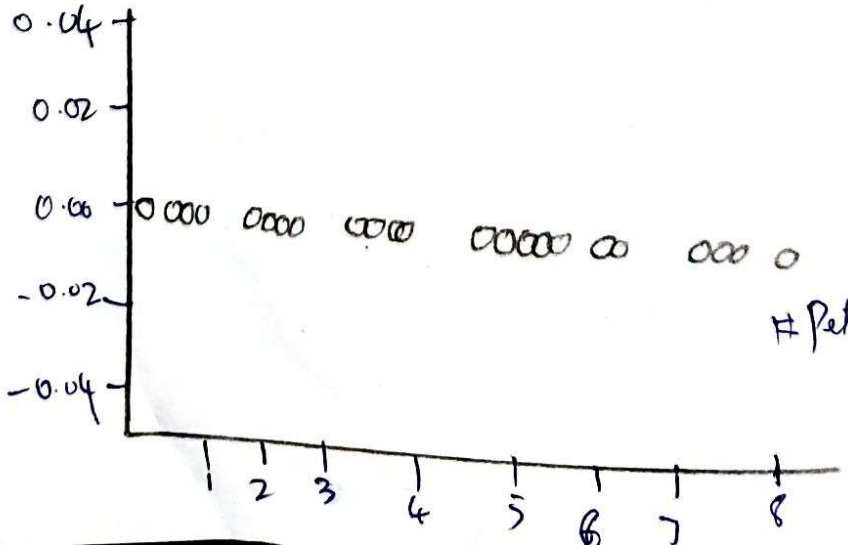
* here we two Independent variables
* Fit a Linear Regression model to data using numpy 'Polyfit' function
* make Prediction and calculate R-Squared value.



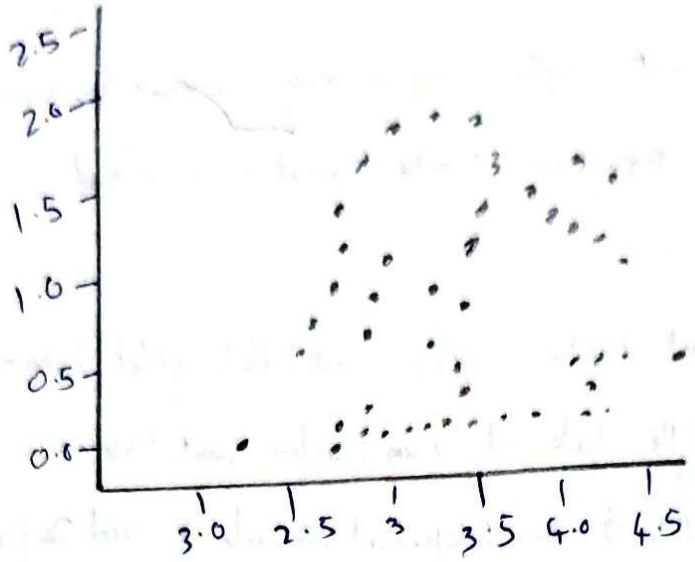
univariate sepal width

UserID Gender Age Estimated

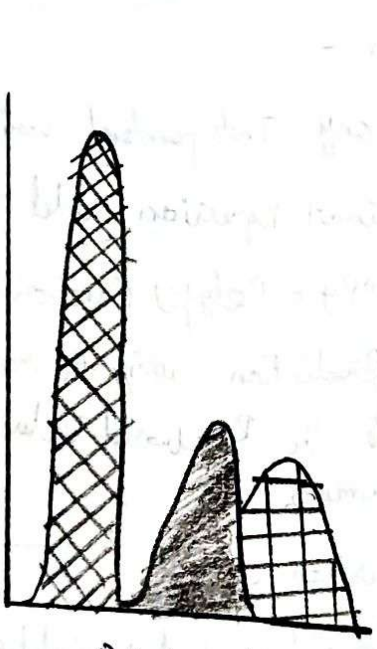




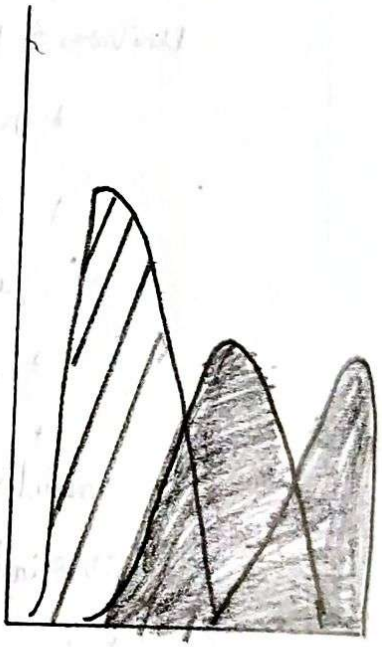
Petal length



bivariate
δ Pal width



Petal width



Sepal length

Multivariate Regression

* Here we more than two variables
* Fit a linear regression model to the data using sklearn's
* Make Prediction and use R-squared value method to calculate model's performance.
* Plot the Result of univariant, bivarent, multivarent Regression using ~~matplotlib~~ matplotlib.Pyplot
  * Print the coefficient (slop)
  * Print the R-squared value for each Regression model

Program:-

Importing Libraries and Reading Data

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
df = pd.read_csv('iris.csv')
df.head(150)
df.shape(150,5)
```

1) univariant for Sepal width

```
df.loc[df['variety']=='Setosa']
df_setosa = df.loc[df['variety']=='Setosa']
df_virginica = df.loc[df['variety']=='Virginica']
df_versicolor = df.loc[df['variety']=='versicolor']
plt.scatter(df_setosa['Sepal.width'],np.zeros_like(
                df_setosa['Sepal.width']))
```

```
plt.scatter(df_virginica['sepal width'],
            np.zeros_like(df_virginica['sepal width']))
plt.scatter(df_versicolor['sepal width'],
            np.zeros_like(df_versicolor['sepal width']))
plt.xlabel('sepal width')
plt.show()
```

2) univariant for sepal length

```
df.loc[df['variety']=='setosa']
df_setosa = df.loc[df['variety']=='setosa']
df_virginica = df.loc[df['variety']=='virginica']
plt.scatter(df_setosa['sepal length'],
            np.zeros_like(df_setosa['sepal length']))
plt.scatter(df_virginica['sepal length'],
            np.zeros_like(df_virginica['sepal length']))
plt.xlabel('sepal length')
plt.show()
```

2) multivariant regression

```
data.columns = ['sepal_length', 'sepal_width',
                'Petal_length', 'Petal_width']
X_bi = data[['sepal_length']]
Y = data['Petal_length']
```

```
X_train_bi , X_test_bi, Y_train_bi,
Y_test_bi = train_test_split (X_bi, Y, test_size=0.3,
random_state = 42)
model_bi = LinearRegression()
model_bi.fit (X_train_bi, Y_train_bi)
Y_pred_bi = model_bi.predict (X, test_bi)
plt.scatter (X_test_bi, Y_test_bi)
plt.plot (X_test_bi, Y_pred_bi, color = 'red')
plt.show()
```

3) Multivariant Regression

```
X_multi = data [['sepal_length', 'sepal_width',
'Petal_width']]
X_train_multi, X_test_multi, Y_train_multi,
Y_test_multi = train_test_split (X_multi, Y,
test_size = 0.3, random_state = 42)
model_multi = LinearRegression()
model_multi.fit (X_train_multi, Y_train_multi)
Y_pred_multi = model_multi.Predict (X_test_multi)
Print (model_multi.coef_)
Print ("mean square error: ", mean_squared_error (
Y_test_multi, Y_pred_multi))
```

Result:-

Thus the Program to implement univariant, bivariant and multivariat regression feature for the given iris dataset is analysed

2) Linear Regression using Least Square method

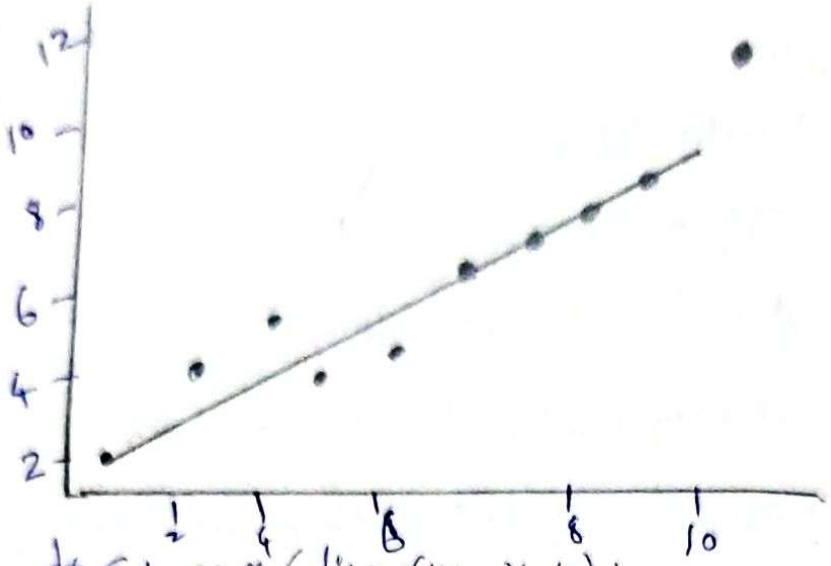
Aim:-

To implement a Python program for constructing a simple linear regression using least Square method

Algorithm:-

* Import Required Libraries (Pandas, matplotlib)
* Read dataset using read-csv() and store in a variable
* Extract independent variable (X) and dependent variable (Y).
* calculate mean of X and Y.
* compute coefficients slop (m) intercept (b)
* Predict value using $y = mx + b$
* Plot scatter points and regression line
* calculate R^2 TSS, RSS, $R^2 = 1 - (RSS/TSS)$
* Print slope, intercept, and R^2
* ~~combine all steps and run the prog~~

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
data = pd.read_csv('headbrain.csv')
X, Y = np.array(list(data['Head Size(cm^3)'])),
       np.array(list(data['Brain Weight(grams)']))
print(X[:5], Y[:5])
[4512]
def get_line(x, y):
    x_m, y_m = np.mean(x), np.mean(y)
    print(x_m, y_m)
    x_d, y_d = x - x_m, y - y_m
    m = np.sum(x_d * y_d) / np.sum(x_d ** 2)
    c = y_m - (m * x_m)
    print(m, c)
    return lambda x: m * x + c
lin = get_line(x, y)
x = np.linspace(np.min(x) - 100, np.max(x) + 100, 1000)
y = np.array([lin(x) for x in x])
plt.plot(x, y, color='red', label='Regression line')
plt.scatter(X, Y, color='green', label='scatter plot')
plt.xlabel('Head Size(cm^3)')
plt.ylabel('Brain weight(grams)')
plt.legend()
plt.show()
```




```
def get_error(line_fue, x, y):
    y_m = np.mean(y)
    y_Pred = np.array([line_fue(_) for _ in x])
    SS_t = np.sum((y-y_m)**2)
    SS_r = np.sum((y-y_Pred)**2)
    return 1-(SS_r/SS_t)
get_error(lin, x, y)
```

In-built Package

```
from sklearn.linear_model import LinearRegression
X = x.reshape((len(x), 1))
reg = LinearRegression()
reg = reg.fit(x, y)
print(reg.score(x, y))
```

	User ID	Gender	Age	Estimated Salary
0	15624	Male	19	19000
1	158109	Male	35	33000
2	156875	Female	26	43000

```
[[-1.05771498  0.53420428]
 [ 1.04385757  0.67576806]]
```

confusion matrix

```
[[31  1]
 [ 1  2]]
```

Accuracy : 0.95

Result :- Thus the classification of simple Logistic Regression using least square method is built successfully.

3) Logistic model

Aim:-

To implement Python Program for the logistic model using any dataset

Algorithm:

Step 1. Import necessary libraries Pandas, sklearn model selection, sklearn preprocessing, sklearn. linear-model matplotlib.pyplot.

Step 2 Read the Data set using pd.read_csv into a data frame

Step 3: preprocess the data, encode the categorical variables and split the data into features (X) target variable (Y)

Step 4:- split the data set into training and testing sets using train-test-split

Step 5:- Standardize the features using standard scaler to ensure they have same scale

Step 6:- Create and train the model using logistic regression and use the formula (1/1+e-z) to return the computed value Invoke standardize() function for "X-train" and "x-test"

Step 7: make predictions using predict() and assign values of testing & training data as "y-pred & "y-train" compute f1-score assign value to "f score" and "f1-score-te"

Step 8: Evaluate the model

Step 9:- Visualize the results

Program:-

```
import pandas as pd
import numpy as np
from numpy import log, dot, exp, shape
from sklearn.metrics import confusion_matrix
data = pd.read_csv("../input/social-network-ads/Social_Network_Ads.csv")
print(data.head())
```

	Age	Salary	Purchased
0	25	50000	0
1	30	60000	0
2	35	63000	1
3	40	70000	1
4	28	52000	0

x = data.iloc[, [2,3]].values

y = data.iloc[:,4] values

In Built Function

from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(x,y,

test_size = 0.10, random_state = 0)

import standard scaler

sc = standard scaler()

x_train = sc.fit_transform(x_train)

x_test = sc.transform(x_test)

Print(x_train[0:10,:])

from sklearn.linear_model import Logistic Regression

classifier = Logistic Regression(random_state = 0)

classifier = .fit(x_train, y_train)

Logistic Regression(random_state = 0)

y_Pred = classifier.Predict(x_test)

Print(y_Pred)

from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_Pred)

Print("confusion matrix : \n", cm)

```
from sklearn.metrics import accuracy_score
print ("Accuracy:", accuracy_score(y_test, y_pred))

conf_mat = confusion_matrix(y_test, y_pred))
accuracy = (conf_mat[0,0] + conf_mat[1,1])/
           sum(sum(conf_mat))
print ("Accuracy is :", accuracy)
```

output

```
[0 1 0 1 1 0 0 1]
confusion matrix :
[[4 1]
 [1 2]]
Accuracy : 0.75
Accuracy % : 75.0
```

Result:-

Thus the Python Program to implement logistic regression for the given SNV-cap, The Performance of the developed model is measured using F1-Score and Accuracy.

4) Python Program to implement single Layer Perceptron

Aim:-

To implement a single Layer Perceptron using Python to Perform binary classification

Procedure:-

+ Initialize weights and bias to small values (or Random)
+ Define an activation function (step function)
+ Provide training data (input and expected outputs)
+ For each epoch
  calculate weighted sum:-
  $y = w_1 w_1 + w_2 w_2 + b$
  apply activation function
+ Update weights using error
  $w = w + \eta \cdot error \cdot x$

Program:-

```
import numpy as np
def step_function(x):
    return 1 if x >=0 else 0
X = np.array([[0,0],[0,1],[1,0],[1,1]])
Y = np.array([0,0,0,1])
weights = np.zeros(2)
bias = 0
learning_rate = 0.1
epochs = 10
```

Output :-

Single Layer Perceptron Result

Accuracy = 0.8

confusion matrix :

[[10 0 0]
[0 7 3]
[0 3 7]]

classification Report

	Precision	Recall	F1-Score	Support
Setosa	1.00	1.00	1.00	10
Versicolor	0.70	0.70	0.70	10
Virginica	0.70	0.70	0.70	10
Accuracy				
macro avg	0.80	0.80	0.80	30
weight avg	0.80	0.80	0.80	30

```
for epoch in range(epochs):
    Print(f"Epoch {epoch+1}")
    for i in range(len(X)):
        linear_output = np.dot(X[i], weights) + bias
        Y_Pred = step_function(linear_output)
        error = Y[i] - Y_Pred
        weights += learning_rate * error * X[i]
        bias += learning_rate * error
        Print(f"Input: {X[i]}, Target:
        {Y[i]}, Predicted: {Y_Pred}")
Print("weights:", weights)
Print("Bias:", bias)
Print("\nTesting:")
for i in range(len(X)):
    output = step_function(np.dot(X[i], weights)+bias)
    Print(f"{X[i]} -> {output}")
```

Result :-

The Perceptron Program for Binary classification is successfully executed and output is verified.

5) MULTI LAYER PERCEPTRON

Aim:-

To implement a multi-layer Perceptron using Backpropagation algorithm for training a neural network

Procedure:-

* Initialize the neural network structure (input, hidden, output layers).
* Assign random weights and biases
* Define activation function (sigmoid)
* Perform forward propagation:
  compute hidden layer output
* calculate error between predicted and actual output
* Perform backpropagation:
  compute gradients
  Update weights using learning rate
* Repeat steps for multiple epochs until error is minimized

Output:

multi Layer Perceptron Results

Accuracy : 0.96666667

confusion matrix:

[[10 0 0]
 [0 9 1]
 [0 0 10]]

classification Report

	Precision	recall	f1-score	support
setos	1.00	1.00	1.00	10
Vericolor	1.00	0.90	0.95	10
Virginica	0.91	1.00	0.95	10
Accuracy			0.97	30
macro-avg	0.97	0.97	0.97	30
weighted avg	0.97	0.97	0.97	30

Program :-

```
import numpy as np
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
def sigmoid_derivative(x):
    return x * (1 - x)
X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([[0],[1],[1],[0]])
np.random.seed(1)
input_neurons = 2
hidden_neurons = 4
output_neurons = 1
wh = np.random.uniform(size=(input_neurons,
    hidden_neurons))
bh = np.random.uniform(size=(1,hidden_neurons))
wo = np.random.uniform(size=(hidden_neurons,
    output_neurons))
bo = np.random.uniform(size=(1,output_neurons))
lr = 0.5
for epoch in range(10000):
    hidden_input = np.dot(X,wh) + bh
    hidden_output = sigmoid(hidden_input)
```

```
final_input = np.dot(hidden_output, wo) + bo
final_output = sigmoid(final_input)
error = Y - final_output
d_output = error * sigmoid_derivative(final_output)
error_hidden = d_output.dot(wo.T)
d_hidden = error_hidden * sigmoid_derivative(hidden_output)
wo += hidden_output.T.dot(d_output) * lr
bo += np.sum(d_output, axis=0, keepdims=True) * lr
wh += X.T.dot(d_hidden) * lr
bh += np.sum(d_hidden, axis=0, keepdims=True) * lr

Print("output after training:")
Print(final_output)
```

Result:-

The multi-Layer Perceptron was successfully trained using the back propagation algorithm

6) Face Recognition using SVM classifier

Aim:-

To develop a face recognition system using SVM classifier in Python.

Procedure:-

* Import required libraries like CV2, numpy and Sklearn.
* Load dataset of face images (labeled).
* Convert images to grayscale.
* Detect faces using Haar cascade.
* Extract features (flatten image or use embeddings)
* Split dataset into training and testing sets
* Train an SVM classifier using training data
* Test the model using test data
* Predict and display recognized faces.

Program:-

```
import cv2
import numpy as np
import os
from sklearn.model_selection import
train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
```

SVM accuracy = 0.93

Classification Report :

	Precision	Recall	f1-score	support
0	0.95	0.91	0.96	10
1	0.95	0.9	0.96	9
2	0.96	0.92	0.96	11
Accuracy			0.95	30
macro avg	0.95	0.95	0.95	30
weighted avg	0.95	0.95	0.95	30

```
face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')

faces, labels = [], []
dataset = "dataset"

for i, person in
    enumerate(os.listdir(dataset)):
        for img_name in
            os.listdir(os.path.join(dataset, person,
                img_name), 0)

        f = face_cascade.detectMultiScale(img, 1.3, 5)
            for (x, y, w, h) in f:
                face = cv2.resize(img[y:y+h, x:x+w),
                    (100, 100))
                faces.append(face.flatten())
                labels.append(i)
    faces, labels = np.array(faces),
    np.array(labels)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    faces, labels, test_size = 0.2)
model = SVC(kernel = 'linear').fit(X_train, y_train)
cap = cv2.VideoCapture(0)
while True:
    ret, frame = cap.read()
    gray = cv2.cvt~~Color~~(frame, cv2.COLOR_BGR)
    f = face_cascade.detectMultiScale(gray, 1.3, 5)
    for (x, y, w, h) in f:
        face = cv2.resize(gray[y:y+h, x:x+w],
        (100, 100)).flatten().reshape(1, -1)
        pred = model.predict(face)
        cv2.putText(frame, str(pred[0]),
        (x, y-10), 1, 1, (0,255,0), 2)
        cv2.rectangle(frame, (x,y), (x+w, y+h),
        (0, 255, 0), 2)
        cv2.imshow("Face", frame)
        if cv2.waitKey(1) == 27: break
    cap.release()
    cv2.destroyAllWindows()
```

Result :-

Thus the above program is executed and output is verified.

7) Decision tree classification

Aim:-

To implement a Decision tree algorithm for classification using Python and evaluate its performance

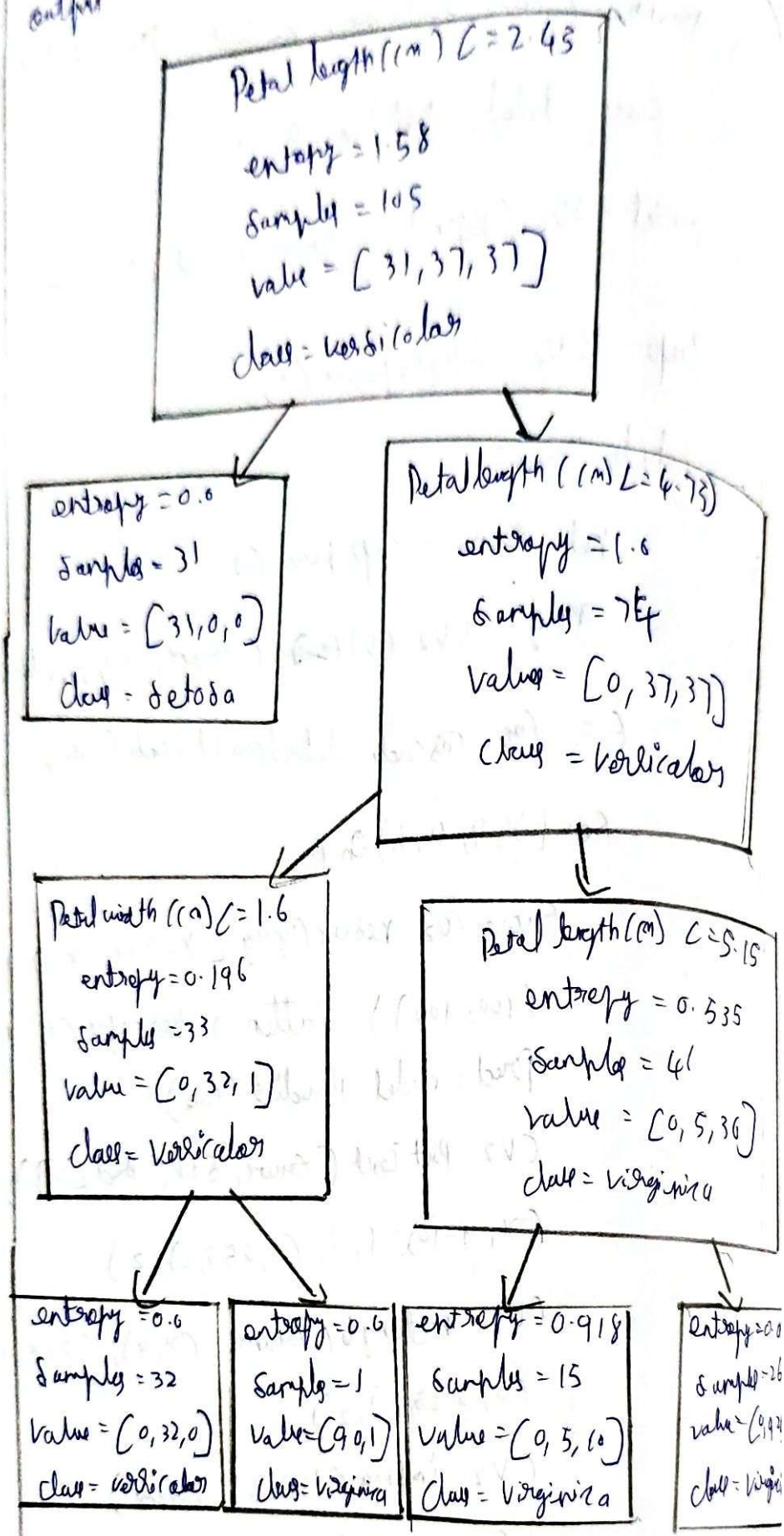
Procedure

1) import required libraries like sklearn, pandas and matplotlib.
2) Load a dataset (Iris dataset)
3) Split the dataset into training and testing sets
4) Create a Decision tree classifier
5) Train the model using training data
6) Predict result using test data
7) Evaluate the model using accuracy score

Program :-

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import test_train_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn import tree
import matplotlib.pyplot as plt.
```

output :-


Petal length (cm) <= 2.45
entropy = 1.58
samples = 105
value = [31, 37, 37]
class = versicolor
entropy = 0.0
samples = 31
value = [31, 0, 0]
class = setosa
Petal length (cm) <= 4.75
entropy = 1.0
samples = 74
value = [0, 37, 37]
class = versicolor
Petal width (cm) <= 1.6
entropy = 0.196
samples = 33
value = [0, 32, 1]
class = versicolor
Petal length (cm) <= 5.15
entropy = 0.535
samples = 41
value = [0, 5, 36]
class = virginica
entropy = 0.0
samples = 32
value = [0, 32, 0]
class = versicolor
entropy = 0.0
samples = 1
value = [0, 0, 1]
class = virginica
entropy = 0.918
samples = 15
value = [0, 5, 10]
class = virginica

```
data = load_iris()
X = data.data
y = data.target
X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.3)
model = DecisionTreeClassifier()
model.fit(~~model~~ X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
Print("Accuracy:", accuracy)

plt.figure(figsize=(10,6))
tree.plot_tree(model, feature_names=data.feature_names,
class_names=data.target_names, filled=True)
plt.show()
```

Result :-

The Decision Tree model was successfully implemented using Python.

8) Boosting (AdaBoost) Algorithm

Aim:-

To implement the Boosting technique using the AdaBoost algorithm in Python and evaluate its performance.

Procedure:-

- import required libraries (Sklearn, Pandas, etc)
- Load a dataset
- Split the dataset into training and testing data
- Create an AdaBoost classifier
- Train the model using training data
- Predict outputs for test data
- Evaluate the model using accuracy score

Program:-

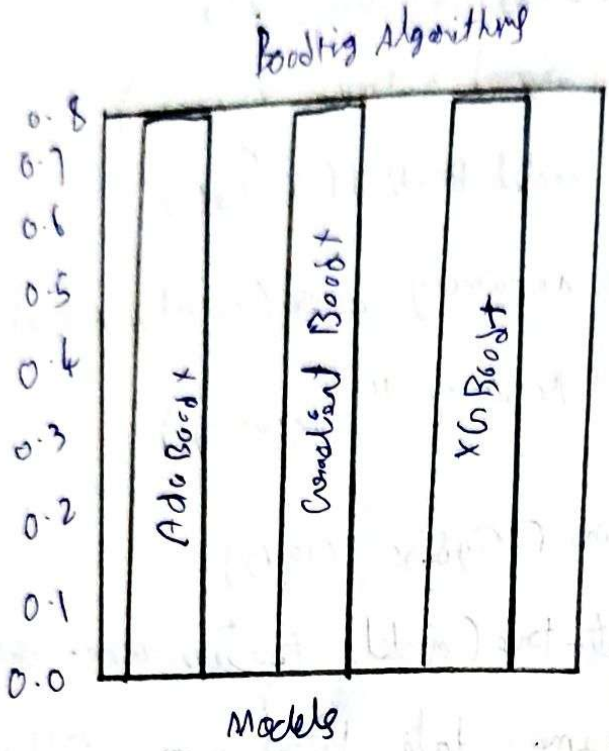
```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score

data = load_iris()
X = data.data
Y = data.target
```

output::

AdaBoost Accuracy : 0.81666667

Gradient Boosting Accuracy : 0.8333331

XGBoost Accuracy : 0.8333334



```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y,
    test_size = 0.3)
model = AdaBoostClassifier(n_estimators = 50, random_state=0)
model.fit(X_train, Y_train)
Y_pred = model.predict(X_test)

accuracy = accuracy_score(Y_test, Y_pred)
print("Accuracy:", accuracy)
# 2. Gradient Boost
gb = GradientBoostingClassifier(n_estimators=30, lr=0.1)
y_pred_gb = gb.predict(X_test)
acc_gb = accuracy_score(y_test, y_pred_gb)

#3)XGBoost
xgb XGBClassifier(n_estimators=30, max_depth=3, seed='integer',
xgb.fit(X_train, y, Train)
y_pred_xgb = xgb.predict(X_test)
acc_xgb = accuracy_score(y_test, y_pred_xgb)
plt.bar(model, accuracies)
plt.xlabel('models')
plt.show
```

Result :-

Thus the Boosting model was successfully implemented in Python.

9) KNN and K-means

Aim :-

To implement KNN classification and K-means clustering in Python

Procedure :-

* Load dataset which contains features and labels
* compute distance between test point and all training points
* Sort distances
* Pick K nearest neighbours
* Perform majority voting
* output predicted class.

Program :-

```
import math

dataset = [
        [2, 4, "A"],
        [4, 6, "A"],
        [6, 8, "B"],
        [8, 10, "B"]
    ]
```

Test Point : [5, 7]

k = 3

point	class	Distance
[2, 4]	4	$\sqrt{(5-2)^2 + (7-4)^2} = \sqrt{18}$ = 4.24
[4, 6]	A	$\sqrt{(5-4)^2 + (7-6)^2} = \sqrt{2}$ 1.41
[6, 8]	B	$\sqrt{(5-6)^2 + (7-8)^2} = \sqrt{2}$ 1.41
[8, 10]	B	$\sqrt{18}$ = 4.24

A = 2 Votes

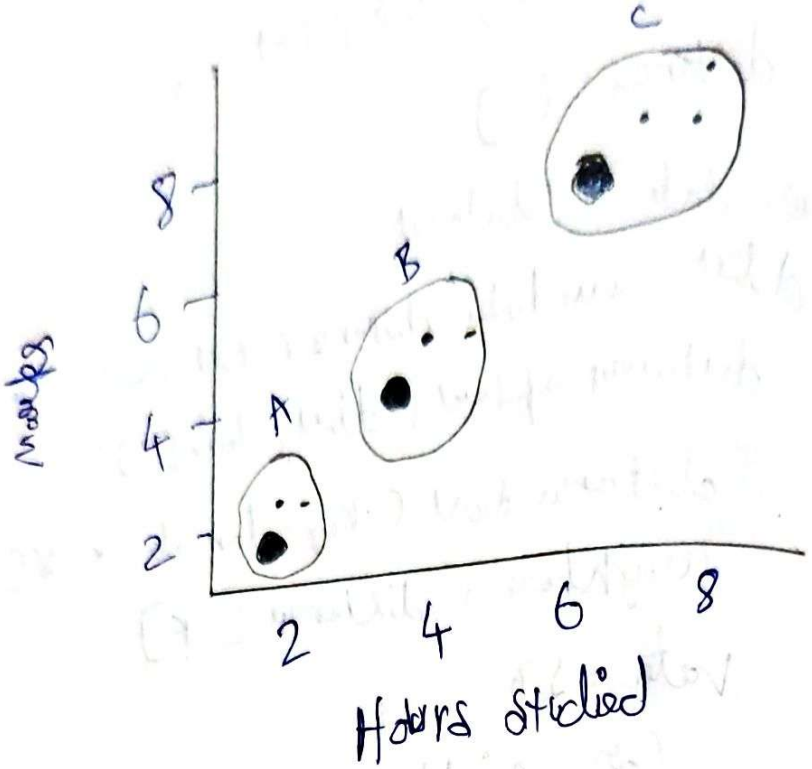
B = 1 Vote

Test Point : [5, 7]

Predicted class : A

```
def euclidean_distance(p1, p2):
    return math.sqrt((p1[0] - p2[0])**2 +
    (p1[1] - p2[1])**2)

def knn(dataset, test_point, k):
    distances = []
    for data in dataset:
        dist = euclidean_distance(test_point, data)
        distances.append((dist, data[2]))
    distances.sort(key = lambda x: x[0])
    neighbors = distances[:k]
    votes = {}
    for neighbor in neighbors:
        label = neighbor[1]
        votes[label] = votes.get(label, 0) + 1
    sorted_votes = sorted(votes.items(),
    key = lambda x: x[1], reverse = True)
    return sorted_votes[0][0]

test_point = [5, 7]
k = 3
result = knn(dataset, test_point, k)
print("Test Point", test_point)
print("Predicted Label:", result)
```

output:-



(ii) K mean clustering

```
from sklearn.datasets import load_iris
from sklearn.cluster import Kmeans
import matplotlib.pyplot as plt
data = load_iris()
X = data.data[:, :2]
kmeans = kmeans(n_clusters=3)
Kmean.fit(X)
labels = kmeans.labels_
Centroid = kmeans.cluster_centers_
plt.scatter(X[:, 0], X[:, 1], c=labels)
plt.scatter(centroids[:, 0], centroids[:, 1],
            color='red')
plt.title("K-means clustering")
plt.show()
```

Result:-

Thus, the KNN algorithm successfully predicted the class for given test point also, K mean clustering algorithm successfully grouped.

10) Dimensionality reduction PCA

Aim:-

To implement dimensionality reduction using Principal component Analysis (PCA)

Procedure

* Import required libraries like Numpy
* create or load dataset
* compute the mean of each feature
* center the data by subtracting the mean
* calculate the covariance matrix
* Find eigenvalues and eigenvectors of the covariance matrix
* Sort eigenvectors in descending order of eigenvalues
* Select top K eigenvectors (Principal components)
* Transform the data onto the new feature space
* obtain the reduced dataset

output :-

original data shape (8,2)

Reduced data shape : (8,1)

Reduced Data :

[[-0.62474644]
 [1.98775751]
 [-0.77533827]
 [-0.63773022]
 [-1.47316262]
 [-0.70119100]
 [0.29781701]
 [1.35254402]]

Program :-

```
import numpy as np

X = np.array([
    [2.5, 2.4],
    [0.5, 0.7],
    [2.2, 2.9],
    [1.9, 2.2],
    [3.1, 3.0],
    [2.3, 2.7],
    [2.0, 1.6], [1.5, 1.6], [1.1, 0.9]])

mean = np.mean(X, axis=0)
X_centered = X - mean
cov_matrix = np.cov(X_centered, rowvar=False)
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)
sorted_indices = np.argsort(eigenvalues)[::-1]
sorted_eigenvectors = eigenvectors[:, sorted_indices]

k = 1
Principal_components = sorted_eigenvectors[:, :k]
X_reduced = np.dot(X_centered, Principal_components)

Print("Original Data shape:", X.shape)
Print("Reduced Data shape:" X_reduced.shape)
Print("Reduced Data :\n", X_reduced)
```